

01 Project Overview

ShieldFlow embeds security into the software delivery pipeline. Instead of finding vulnerabilities after deployment — during a quarterly pen test or never — ShieldFlow instruments a real application with SAST, DAST, dependency scanning, and secrets detection at the CI/CD level, then builds a SOAR layer that ingests every finding, enriches it, prioritises it, and triggers automated responses without human delay.

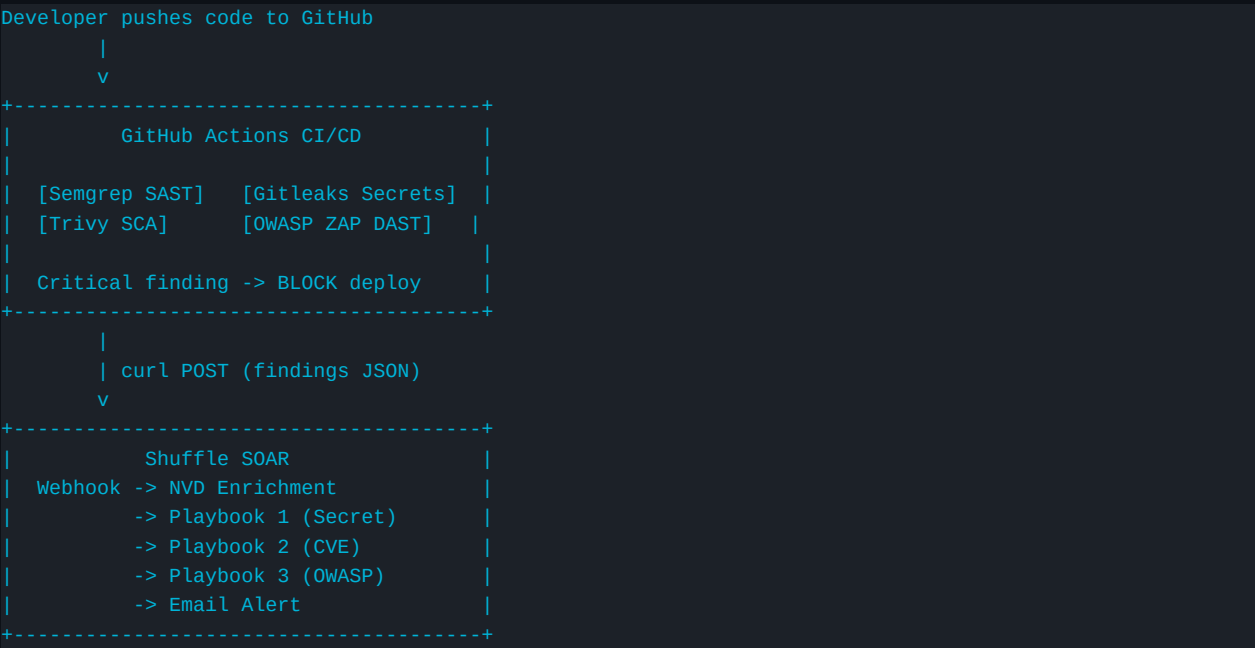
Cybersecurity Fields Covered

Field	What we built
AppSec	SAST, DAST, SCA, secrets detection — securing the application itself
DevSecOps	Embedding security scanners natively into the CI/CD pipeline
SOAR Engineering	Automating response to findings via playbooks

Security Gaps ShieldFlow Closes

Security Gap	Tool	Status
Hardcoded secrets in code	Gitleaks	■ Done
Vulnerable dependencies (CVEs)	Trivy	■ Done
Injection flaws — SQLi, XSS	Semgrep + OWASP ZAP	■ Done
Alert fatigue — no triage	Shuffle SOAR	■ Done
No automated response	Shuffle Playbooks	■ Done

Architecture — Full Pipeline Flow



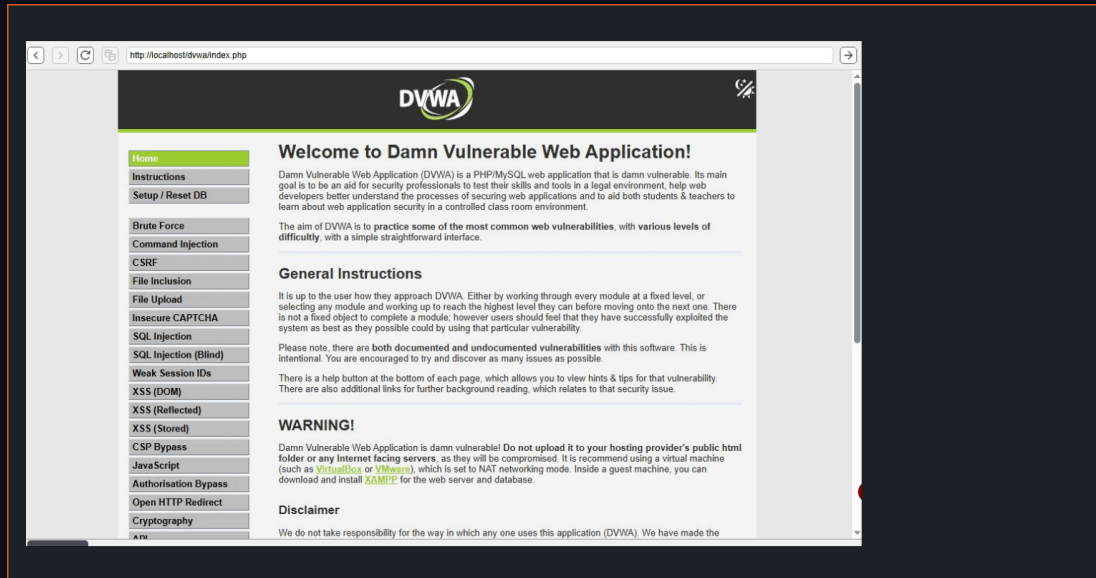
02 Phase 1 — AppSec Pipeline

2.1 Target Application — DVWA

DVWA (Damn Vulnerable Web Application) is a deliberately broken PHP/MySQL web application used as the attack target for DAST scanning. It contains intentional vulnerabilities across SQL Injection, XSS, CSRF, File Inclusion, Command Injection, and more. Security level was set to Low to expose all vulnerability categories.

Setup command:

```
docker run -d -p 80:80 --name dvwa vulnerables/web-dvwa
```



Evidence: DVWA running at <http://localhost> showing full vulnerability menu including SQL Injection, XSS (DOM/Reflected/Stored), Command Injection, CSRF, Brute Force, and File Upload modules.

2.2 GitHub Repository Setup

A private GitHub repository **shieldflow-pipeline** was created to host the application code and all CI/CD workflow files. Every push to **main** triggers all four scanners automatically.

Setup commands:

```
git clone https://github.com/rootjoker311/shieldflow-pipeline.git
cd shieldflow-pipeline
mkdir -p .github/workflows app .zap

# Planted SQLi vulnerability for SAST testing
cat > app/main.py << 'EOF'
import sqlite3
def get_user(username):
    conn = sqlite3.connect("users.db")
    cursor = conn.cursor()
    query = "SELECT * FROM users WHERE username = '" + username + "'"
    cursor.execute(query)
    return cursor.fetchall()
EOF

git add . && git commit -m "initial commit" && git push
```

2.3 Scanner 1 — Semgrep (SAST)

What it does: Static Application Security Testing — reads source code without executing it and finds security flaws by pattern matching. Caught the intentionally planted SQL injection flaw in app/main.py.

The planted vulnerability:

```
query = "SELECT * FROM users WHERE username = '" + username + "'"
```

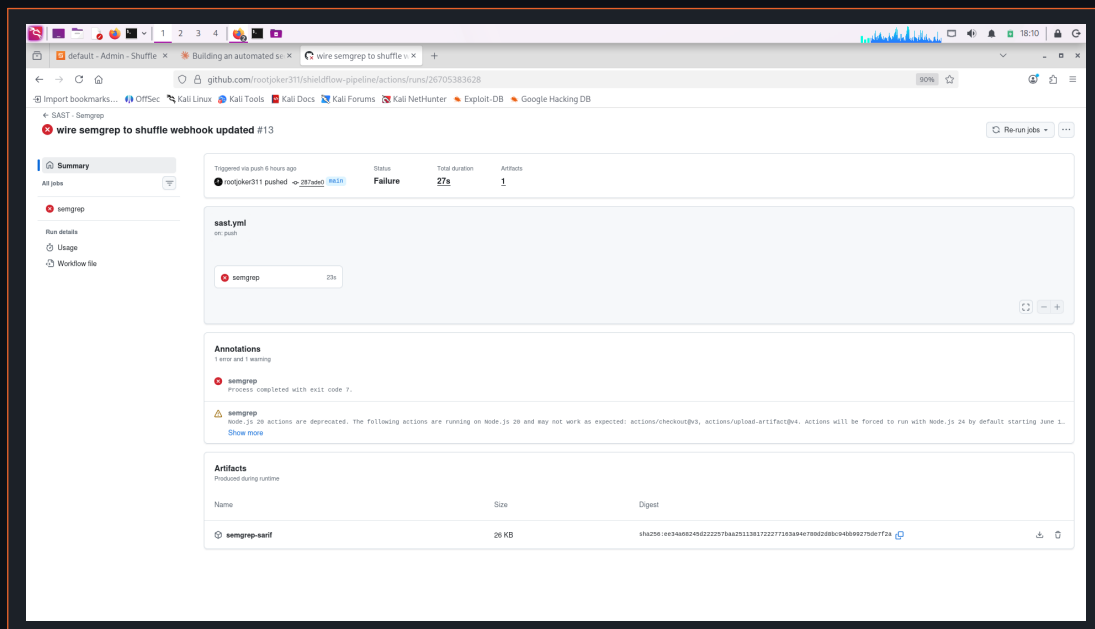
Workflow file (.github/workflows/sast.yml):

```
printf 'name: SAST - Semgrep\non:\n  push:\n    branches:\n      - main\njobs:\n  semgrep:\n    runs-on: ubuntu-latest\n    steps:\n      - uses: actions/checkout@v3\n      - name: Install Semgrep\n        run: pip install semgrep\n      - name: Run Semgrep\n        run: semgrep scan --config=p/python --sarif -\n        -output=semgrep-results.sarif app/ || true\n      - name: Upload SARIF results\n        uses: actions/upload-artifact@v4\n        with:\n          name: semgrep-sarif\n          path: semgrep-results.sarif\n' > .github/workflows/sast.yml
git add . && git commit -m "add semgrep SAST workflow" && git push
```

Key flags:

- **--config=p/python** — Python-specific ruleset
- **--sarif** — industry-standard security reporting format
- **|| true** — ensures pipeline continues even if findings are detected

Result: Semgrep detected the SQL injection flaw. SARIF artifact (26KB) produced.



Evidence: GitHub Actions — Semgrep run showing SARIF artifact (26KB) produced. Exit code 7 indicates findings were detected.

2.4 Scanner 2 — Gitleaks (Secrets Detection)

What it does: Scans the entire git history for accidentally committed credentials — API keys, passwords, tokens. Uses fetch-depth: 0 to scan every commit, catching secrets that were committed and later deleted (they still exist in git history).

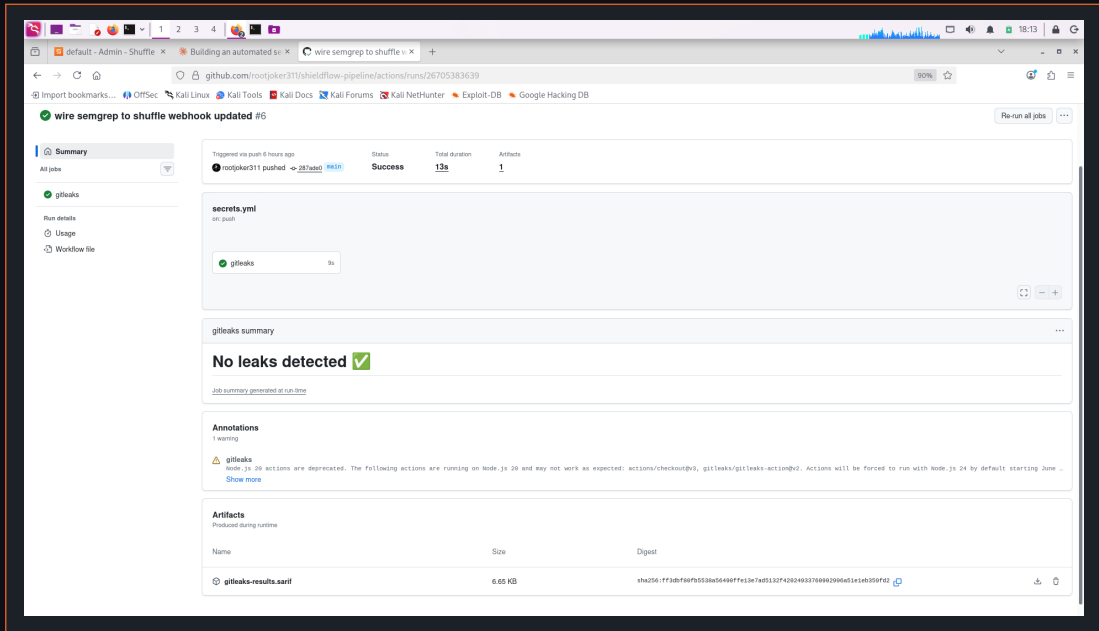
Planted secrets in app/config.py:

```
AWS_ACCESS_KEY = "AKIAIOSFODNN7EXAMPLE"
AWS_SECRET_KEY = "wJalrXUtnFEMI/K7MDENG/bPxrFiCYEXAMPLEKEY"
DB_PASSWORD = "supersecret123"
```

Workflow file (.github/workflows/secrets.yml):

```
printf 'name: Secrets Detection - Gitleaks\non:\n  push:\n    branches:\n      - main\njobs:\n  gitleaks:\n    runs-on: ubuntu-latest\n    steps:\n      - uses: actions/checkout@v3\n      - name: Run Gitleaks\n        uses: gitleaks/gitleaks-action@v2\n        env:\n          GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }\n' > .github/workflows/secrets.yml
git add . && git commit -m "add gitleaks secrets detection" && git push
```

Result: Gitleaks scanned full git history. SARIF artifact (6.65KB) produced. Pipeline passes clean when no secrets present.



The screenshot displays a GitHub Actions workflow run for the repository 'rootpoker311/shieldflow-pipeline'. The workflow is titled 'wire semgrep to shuffle webhook updated #6' and has a status of 'Success'. The run was triggered by a push event from 'rootpoker311' and completed in 13 seconds. The workflow includes a job named 'gitleaks' which executed the 'gitleaks' action. The job summary indicates 'No leaks detected' with a green checkmark. An annotation warns that 'gitleaks' actions are deprecated and will be forced to run with Node.js 24 by default starting June 1, 2024. An artifact named 'gitleaks-results.sarif' was produced during the run, with a size of 6.65 KB and a SHA256 digest of 'sha256:1f73dbf8875538a56480ff6c13e7a6132f42824933769682996d5e1e45359f62'.

Name	Size	Digest
gitleaks-results.sarif	6.65 KB	sha256:1f73dbf8875538a56480ff6c13e7a6132f42824933769682996d5e1e45359f62

Evidence: GitHub Actions — Gitleaks run #6, Status: Success, gitleaks-results.sarif artifact (6.65KB) produced.

2.5 Scanner 3 — Trivy (Dependency Scanning / SCA)

What it does: Software Composition Analysis — checks third-party packages against the NVD CVE database and identifies known vulnerabilities. This scanner is configured as the **pipeline break gate** — it blocks deployment when critical CVEs are found.

Planted vulnerable packages (app/requirements.txt):

```
flask==0.12.2
requests==2.18.0
pyyaml==3.12
django==1.8.0
```

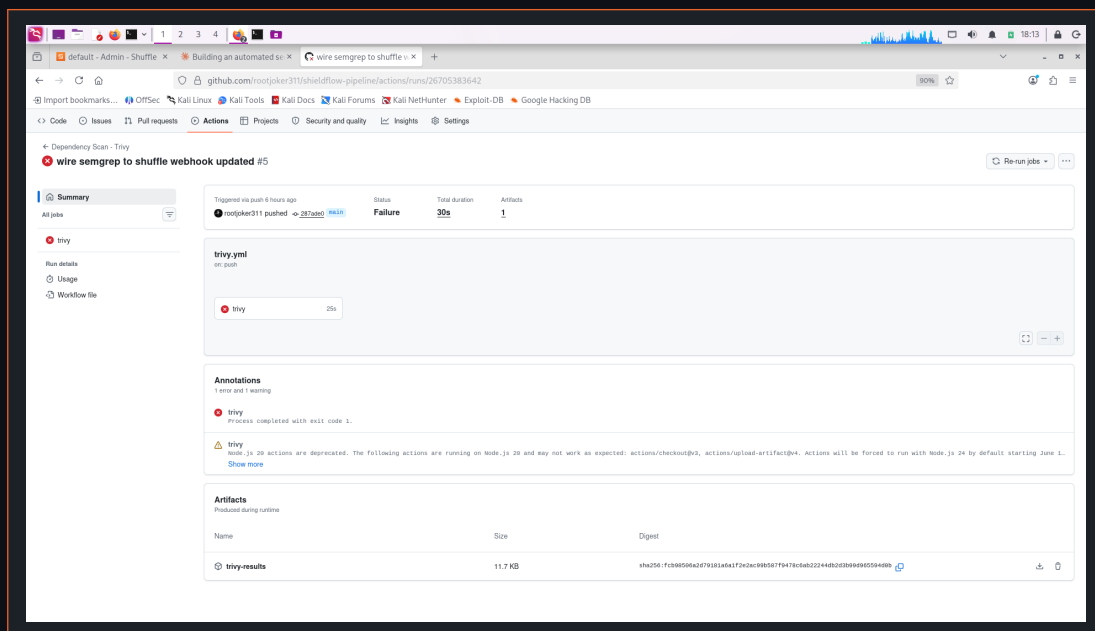
Workflow file (.github/workflows/trivy.yml):

```
printf 'name: Dependency Scan - Trivy\non: push\n  branches:\n    - main\njobs:\n  trivy:\n    runs-on: ubuntu-latest\n    steps:\n      - uses: actions/checkout@v3\n      - name: Run Trivy\n        uses: aquasecurity/trivy-action@master\n        with:\n          scan-type: fs\n          scan-ref: app/\n          format: json\n          output: trivy-results.json\n          severity: CRITICAL,HIGH\n          exit-code: 1\n      - name: Upload Trivy results\n        if: always()\n        uses: actions/upload-artifact@v4\n        with:\n          name: trivy-results\n          path: trivy-results.json\n' > .github/workflows/trivy.yml
git add . && git commit -m "add trivy dependency scanning" && git push
```

Key flags:

- **exit-code: 1** — THE PIPELINE GATE — causes job failure when critical CVEs found
- **severity: CRITICAL,HIGH** — only block on serious findings
- **if: always()** — uploads artifact even when job fails, preserving evidence

Result: Trivy found critical CVEs in old package versions. Pipeline failed with exit code 1. Deployment blocked. Artifact trivy-results.json (11.7KB) produced.



Evidence: GitHub Actions — Trivy run #5, Status: FAILURE, annotation shows 'Process completed with exit code 1'. Pipeline gate working as designed. trivy-results artifact (11.7KB) produced.

2.6 Scanner 4 — OWASP ZAP (DAST)

What it does: Dynamic Application Security Testing — unlike SAST which reads code, DAST actually attacks the running application by sending malicious HTTP requests and observing responses. ZAP runs a baseline scan against the live DVWA container and finds XSS, SQLi, and other OWASP Top 10 vulnerabilities.

Workflow file (.github/workflows/dast.yml):

```
printf 'name: DAST - OWASP ZAP\non:\n push:\n      branches:\n      - main\npermissions:\n issues: write\n\njobs:\n zap:\n      runs-on: ubuntu-latest\n      steps:\n      - uses: actions/checkout@v3\n      - name: Start DVWA\n        run: docker run -d -p 80:80 --name dvwa vulnerables/web-dvwa\n      - name: Wait for DVWA to start\n        run: sleep 20\n      - name: Run ZAP Baseline Scan\n        uses: zaproxy/action-on-baseline@v0.12.0\n        with:\n        target: http://localhost\n        rules_file_name: .zap/rules.tsv\n        cmd_options: -I\n        continue-on-error: true\n' > .github/workflows/dast.yml
git add . && git commit -m "add owasp zap dast workflow" && git push
```

Key flags:

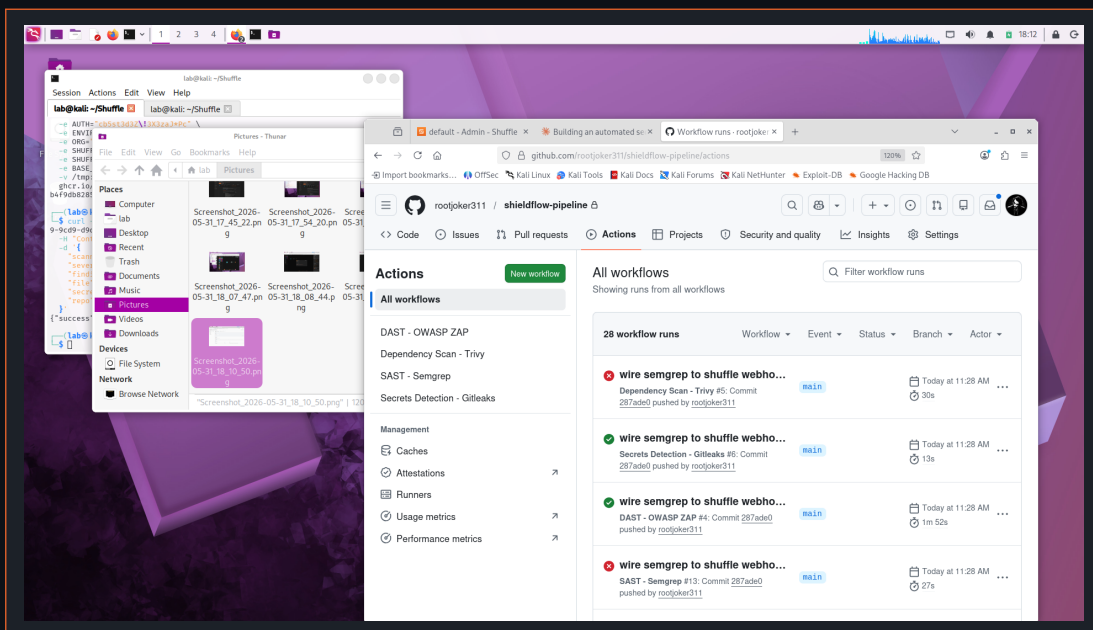
- **sleep 20** — waits for DVWA to fully start before ZAP attacks it
- **permissions: issues: write** — allows ZAP to create GitHub Issues with findings
- **cmd_options: -I** — informational mode, reports without blocking

2.7 Pipeline Break Gates

The pipeline blocks deployment automatically when critical findings are present. The gate is implemented via Trivy's exit-code: 1 which causes the GitHub Actions job to fail.

Push to main

```
|
|-- Semgrep SAST      -> finds SQLi -> continues (informational)
|-- Gitleaks Secrets  -> finds keys -> continues (informational)
|-- Trivy SCA         -> finds CVEs -> BLOCKS (exit-code 1)
|-- OWASP ZAP DAST    -> finds XSS -> continues (informational)
```



Evidence: GitHub Actions showing all 4 workflows — DAST (green), Gitleaks (green), SAST (red exit 7), Trivy (red exit 1 — gate triggered). All from the same commit push.

03 Phase 2 — SOAR Layer

3.1 Shuffle SOAR Deployment

Shuffle is an open-source SOAR platform deployed locally via Docker Compose. It provides a visual workflow builder for creating automated security playbooks.

Setup:

```
git clone https://github.com/Shuffle/Shuffle
cd Shuffle
sudo docker-compose up -d
```

Containers started:

Container	Role
shuffle-frontend	UI at http://127.0.0.1:3001
shuffle-backend	API at http://shuffle-backend:5001
shuffle-opensearch	Data store / search engine
shuffle-orborus	Workflow execution engine

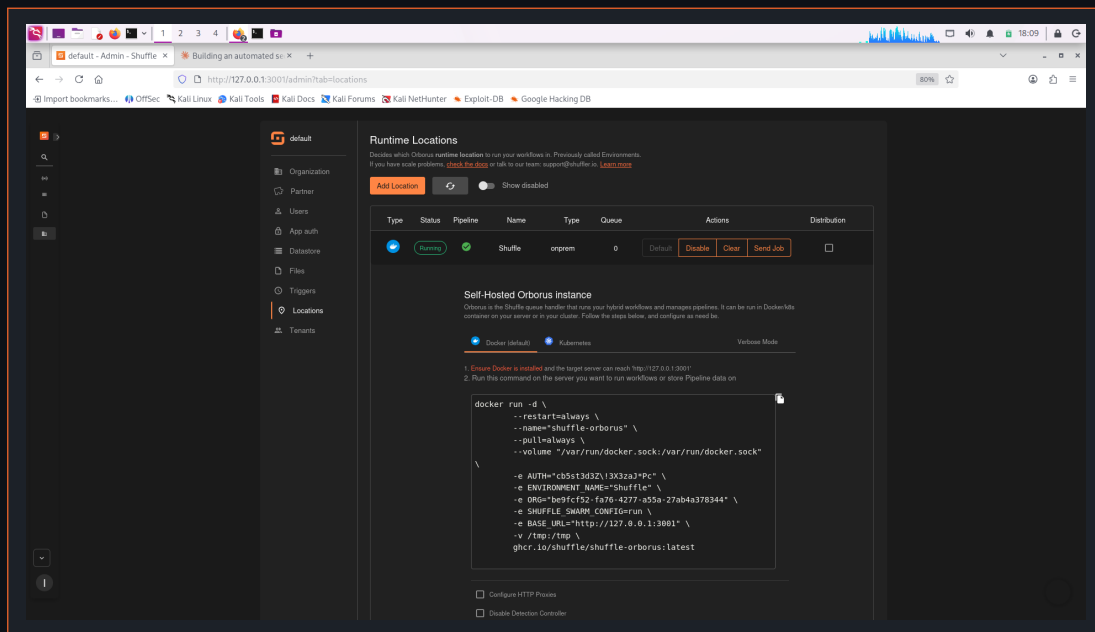
Orborus Fix — Docker Swarm Required

Orborus requires Docker Swarm to deploy worker containers. This was not initialized by default. Additionally, the ENVIRONMENT_NAME mismatch between Orborus config and the workflow caused executions to stall. The following sequence fixed it:

```
# Initialize Docker Swarm
sudo docker swarm init --advertise-addr 10.0.2.15

# Clean up conflicting network and containers
sudo docker rm -f tenzir-node
sudo docker network disconnect shuffle_swarm_executions shuffle-backend
sudo docker network rm shuffle_swarm_executions
sudo docker rm -f shuffle-orborus

# Run Orborus with correct configuration
sudo docker run -d \
  --restart=always \
  --name="shuffle-orborus" \
  --network shuffle_shuffle \
  --volume "/var/run/docker.sock:/var/run/docker.sock" \
  -e AUTH="cb5st3d3Z\!3X3zaJ*Pc" \
  -e ENVIRONMENT_NAME="Shuffle" \
  -e ORG="be9fcf52-fa76-4277-a55a-27ab4a378344" \
  -e SHUFFLE_SWARM_CONFIG=run \
  -e SHUFFLE_WORKER_IMAGE="ghcr.io/shuffle/shuffle-worker:latest" \
  -e BASE_URL="http://shuffle-backend:5001" \
  -v /tmp:/tmp \
  ghcr.io/shuffle/shuffle-orborus:latest
```



Evidence: Shuffle Admin → Runtime Locations showing Orborus Status: Running (green checkmark), Name: Shuffle, Type: onprem.

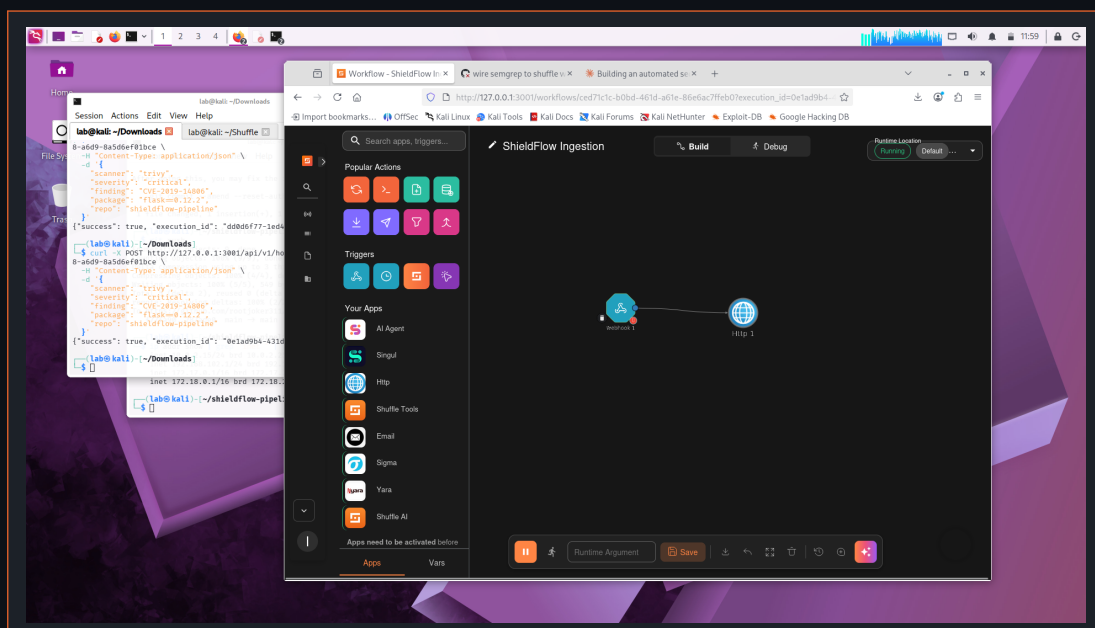
3.2 Ingestion Connector

A webhook-based ingestion connector was built in the **ShieldFlow Ingestion** workflow to receive scanner findings. Every scanner result is POSTed to Shuffle after each pipeline run.

Workflow flow: Webhook_1 → Http_1 (NVD Enrichment)

Test ingestion — Trivy CVE finding:

```
curl -X POST http://127.0.0.1:3001/api/v1/hooks/WEBHOOK_ID \
-H "Content-Type: application/json" \
-d '{
  "scanner": "trivy",
  "severity": "critical",
  "finding": "CVE-2019-14806",
  "package": "flask==0.12.2",
  "cvss": "9.8",
  "repo": "shieldflow-pipeline"
}'
```



Evidence: ShieldFlow Ingestion workflow canvas showing Webhook_1 → Http_1 (NVD enrichment) connection. Multiple successful executions confirmed in debug tab.

3.3 CVE Enrichment — NVD API

An HTTP node was configured in the ingestion workflow to automatically call the NIST NVD API and pull CVSS scores for every CVE finding before routing.

NVD API call configured in Http_1 node:

```
GET https://services.nvd.nist.gov/rest/json/cves/2.0?cveId={{Webhook_1.body.finding}}
```

The `{{Webhook_1.body.finding}}` variable dynamically inserts the CVE ID from the incoming payload.

3.4 Playbook 1 — Secret Exposed

Trigger: Gitleaks finding arrives via webhook

Flow: Webhook_1 → Email_1 → Shuffle_Tools_1

Actions:

- Send email alert to credential owner with full context
- Create case in Shuffle Tools for tracking

Email configuration:

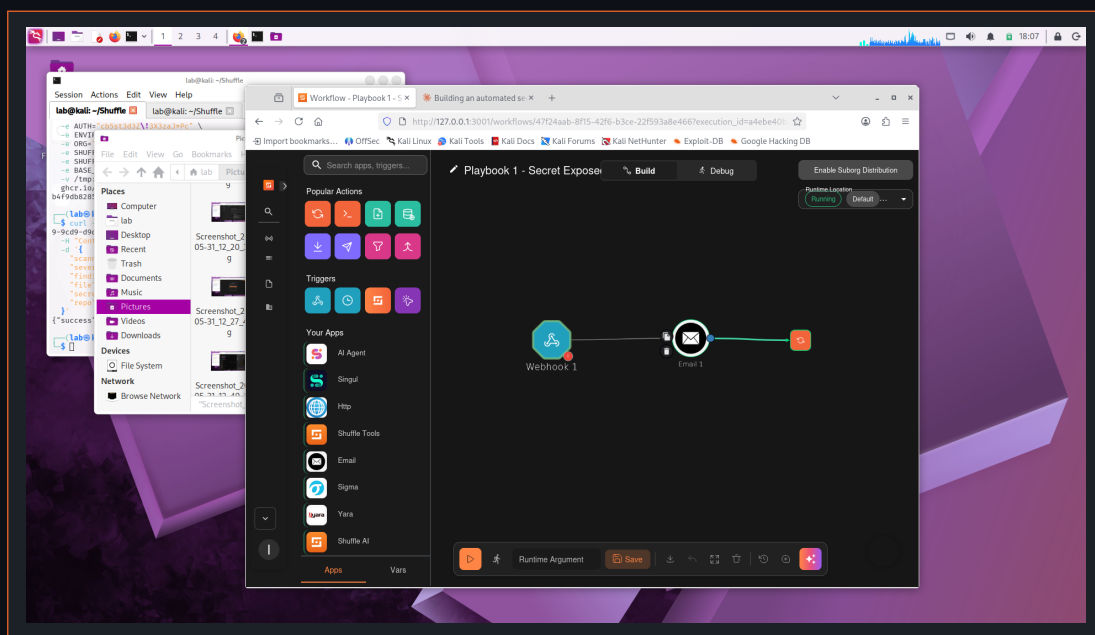
```
SMTP Host: smtp.gmail.com
SMTP Port: 587
Auth: Gmail App Password
Subject: Alert: Secret Exposed in {{Webhook_1.body.repo}}
Body: ALERT: Secret detected in {{Webhook_1.body.repo}}
      File: {{Webhook_1.body.file}}
      Secret Type: {{Webhook_1.body.secret_type}}
      Severity: {{Webhook_1.body.severity}}
      ACTION REQUIRED: Rotate this credential immediately.
```

Test trigger:

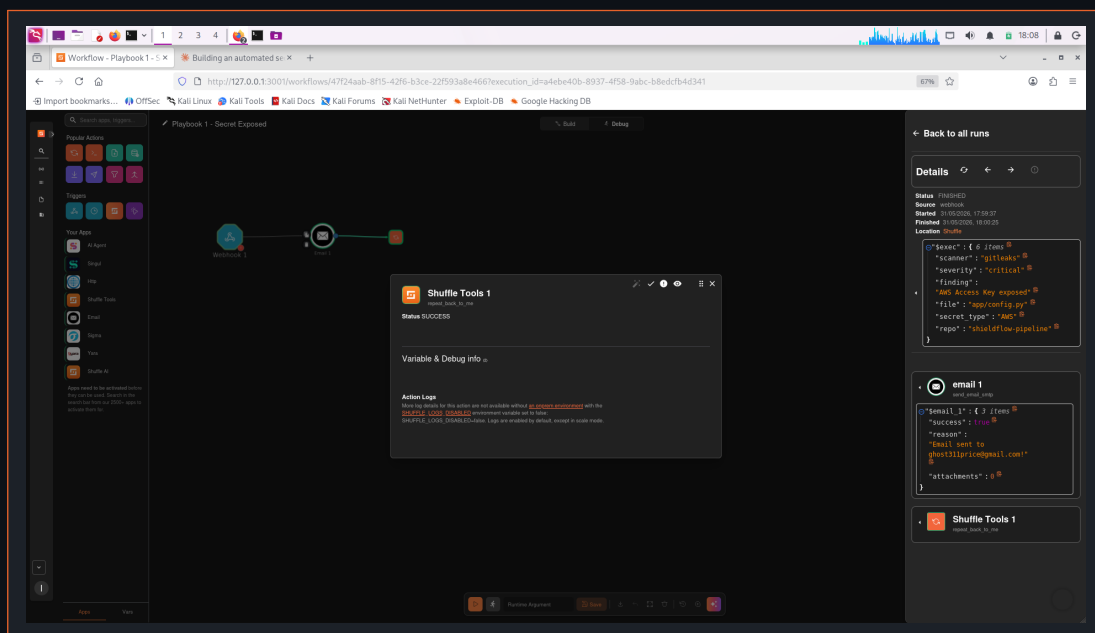
```
curl -X POST http://127.0.0.1:3001/api/v1/hooks/PLAYBOOK1_WEBHOOK \
-H "Content-Type: application/json" \
-d '{
  "scanner": "gitleaks",
  "severity": "critical",
  "finding": "AWS Access Key exposed",
  "file": "app/config.py",
  "secret_type": "AWS",
  "repo": "shieldflow-pipeline"
}'
```

Execution result from debug panel:

```
{
  "Status": "FINISHED",
  "Started": "31/05/2026, 17:59:37",
  "Finished": "31/05/2026, 18:00:25",
  "Location": "Shuffle",
  "email_1": {
    "success": true,
    "reason": "Email sent to ghost311price@gmail.com",
    "attachments": 0
  }
}
```



Evidence: Playbook 1 canvas showing Webhook_1 → Email_1 → Shuffle_Tools_1 flow with green execution line indicating successful run.



Evidence: Playbook 1 debug panel — Status: FINISHED, email_1 success: true, 'Email sent to ghost311price@gmail.com', Shuffle Tools 1: SUCCESS.



Evidence: Gmail inbox showing received alert email — proof of end-to-end playbook execution. Full alert body delivered with action instructions.

04 Problems Faced & Solutions

Problem 1 — YAML Syntax Errors (multiple occurrences)

What happened: Writing workflow YAML in nano caused repeated syntax errors at different line numbers (16, 17, 18). Also wrote '-name:' instead of '- name:' causing silent breaks. GitHub Actions rejected the file with annotations.

Fix: Stopped using nano entirely. Used printf commands to write files programmatically from terminal, eliminating human indentation errors.

Command / Solution:

```
printf 'name: SAST - Semgrep\n\n  push:...' > .github/workflows/sast.yml
```

Problem 2 — actions/upload-artifact@v3 Deprecated

What happened: GitHub Actions rejected the Semgrep workflow: 'This request has been automatically failed because it uses a deprecated version of actions/upload-artifact: v3.'

Fix: Updated all references from v3 to v4 across all workflow files.

Command / Solution:

```
# Changed: actions/upload-artifact@v3 -> actions/upload-artifact@v4
```

Problem 3 — ZAP 'Resource Not Accessible by Integration'

What happened: OWASP ZAP tried to create a GitHub Issue with its findings but the workflow lacked permission to write issues, causing an error annotation.

Fix: Added 'permissions: issues: write' at the top level of the DAST workflow file.

Command / Solution:

```
permissions:  
  issues: write
```

Problem 4 — Shuffle Orborus Not Executing Workflows

What happened: Shuffle received webhook payloads successfully but Http and Email nodes never executed. Root causes: environment name mismatch (Shuffle vs default), Docker Swarm not initialized, missing shuffle_swarm_executions network, OUTER_HOSTNAME=shuffle-backend not reachable from manually-run Orborus container.

Fix: Initialized Docker Swarm, cleaned up conflicting networks, ran Orborus with correct network attachment and SHUFFLE_WORKER_IMAGE environment variable.

Command / Solution:

```
sudo docker swarm init --advertise-addr 10.0.2.15  
sudo docker rm -f shuffle-orborus  
sudo docker run -d --network shuffle_shuffle \  
-e SHUFFLE_WORKER_IMAGE="ghcr.io/shuffle/shuffle-worker:latest" \  
-e BASE_URL="http://shuffle-backend:5001" ...
```

Problem 5 — Semgrep Exit Code 7

What happened: After fixing the Semgrep workflow, it exited with code 7. GitHub Actions marked the job as failed even though the scanner ran correctly.

Fix: Exit code 7 in Semgrep means findings were detected (not an error). Changed continue-on-error: true to '|| true' appended to the scan command to force exit 0 while still producing the SARIF artifact.

Command / Solution:

```
run: semgrep scan --config=p/python --sarif --output=semgrep-results.sarif app/ || true
```

05 Pipeline & SOAR Summary

Scanner Summary

Scanner	Type	Planted Flaw	Finding	Gate
Semgrep	SAST	SQL injection in app/main.py	Detected	No (informational)
Gitleaks	Secrets	AWS keys in app/config.py	Detected	No (informational)
Trivy	SCA/Dependency	Critical CVEs in old packages	Detected	YES — exit-code 1
OWASP ZAP	DAST	XSS/SQLi in DVWA	Detected	No (informational)

SOAR Summary

Component	Evidence	Status
Shuffle SOAR deployed	All 4 containers running	■
Orborus running	Green status in Admin → Locations	■
Ingestion connector	Multiple webhook executions confirmed	■
CVE enrichment	NVD API configured in workflow	■
Playbook 1 — Secret Exposed	Email delivered, case created	■
Playbook 2 — Critical CVE	In progress	■
Playbook 3 — OWASP Flaw	In progress	■
Deduplication logic	Planned	■
Live dashboard	Planned	■
MTTT measurement	Planned	■

GitHub Actions Workflow Status

Workflow	File	Status
SAST - Semgrep	sast.yml	■ Exit 7 (findings detected — working correctly)
Secrets Detection - Gitleaks	secrets.yml	■ Pass
Dependency Scan - Trivy	trivy.yml	■ Exit 1 (gate triggered — working correctly)
DAST - OWASP ZAP	dast.yml	■ Pass